

Implementing Deep Learning and CNNs for Vehicle Detection to Alleviate Highway Traffic Congestion

Haider Adil Khaleel ALzubaidy
Altinbaş University
Graduate School of Science and
Engineering

Electrical And Computer Engineering
Turkey -Istanbul 2023
213720158@ogr.altinbas.edu.tr

Abdullahi Abdu Ibrahim
Altinbaş University
Graduate School of Science and
Engineering

Electrical And Computer Engineering
Turkey -Istanbul 2023
abdullahi.ibrahim@altinbas.edu.tr

Abstract— Traffic jams are a common problem in urban areas, and detecting them in real time can help alleviate congestion and improve traffic flow. This project, propose CNN-based method for detecting traffic jams from live video feeds. Our system involves capturing video streams from cameras placed in strategic locations and processing them using a deep-learning model. The study aims to develop a system for detecting traffic jams using (CNN) Convolutional Neural Networks. The system analyzes real-time traffic images to identify congested areas and provides insights to traffic management authorities for timely intervention. The study examines the accuracy of the system by testing it on a dataset of traffic videos. We evaluated our system on a dataset of traffic videos and obtained an average accuracy 89.05.11%, the error average 0.10%., and average F-score 89.61% indicating that our approach can effectively detect traffic jams. Our experiments show that our CNN-based approach outperforms traditional methods of traffic jam detection and can provide accurate and real-time information to drivers and traffic management systems.

Keywords- Traffic jam, Deep Learning, CNN, Vehicles Detection.

Introduction

Traffic jam on highways is a pressing issue that affects the efficiency of transportation systems and impacts the economy, environment, and quality of life in urban areas. Traditional traffic management methods have limitations in accurately detecting and managing traffic flow on highways in real-time. However, recent advancements in deep learning, specifically (CNNs) Convolutional Neural Networks, offer promising solutions for vehicle detection and traffic jams on highways[1]. One approach is to deploy traffic cameras along highways that capture video streams of the traffic. These video streams can be fed into CNN-based vehicle detection systems, which can accurately detect and track vehicles in real-time. By analyzing the data from these systems.

The process of vehicle detection using CNNs entails training a CNN model on a dataset of labeled images that include both vehicles and non-vehicles. The model learns to differentiate between visual features that are unique to vehicles and those found in other objects in the scene. After training, the model can be applied to detect vehicles in new images or videos by processing the input data through the layers of the network and generating a binary output indicating whether a vehicle is present or not in the image. CNN is a deep learning algorithm

utilized for image and video processing applications, such as object recognition, detection, and segmentation. It is a type of neural network that has become widely used in the field of computer vision [2]. CNNs are made up of multiple layers that perform convolutional and pooling operations, which are then followed by fully connected layers and activation functions. This is the basic architecture of a typical CNN [3]. The presence of these layers within a CNN allows the network to effectively learn complex features and patterns from the input data, which can then be used for classification or regression tasks..

I. PROBLEM STATEMENT

The proposal aims to leverage deep learning-based vehicle detection using CNNs to solve traffic congestion on highways. The primary objectives are to accurately detect and track vehicles in real-time, analyze traffic flow patterns, and implement proactive measures to mitigate traffic congestion.

II. LITERATURE REVIEW

The use of CNNs for vehicle detection using deep learning has become prevalent in various traffic management applications. For instance, Chen et al. (2021) developed a CNN-based vehicle detection system that could effectively detect and track vehicles in real-time, providing accurate traffic information to traffic management authorities[4]. Wang et al. (2020) proposed a deep learning-based vehicle detection system that could identify and classify various types of vehicles accurately, thereby enabling advanced traffic management strategies[5].

III. METHODOLOGY

The proposed system utilizes CNN-based vehicle detection models to analyze video streams from traffic videos, accurately identifying and tracking vehicles in real-time. The collected data is fed into predictive traffic models that forecast traffic congestion patterns and suggest proactive measures to mitigate congestion. The system also provides real-time traffic information to drivers through variable message signs or mobile applications, encouraging route changes and preventing congestion.

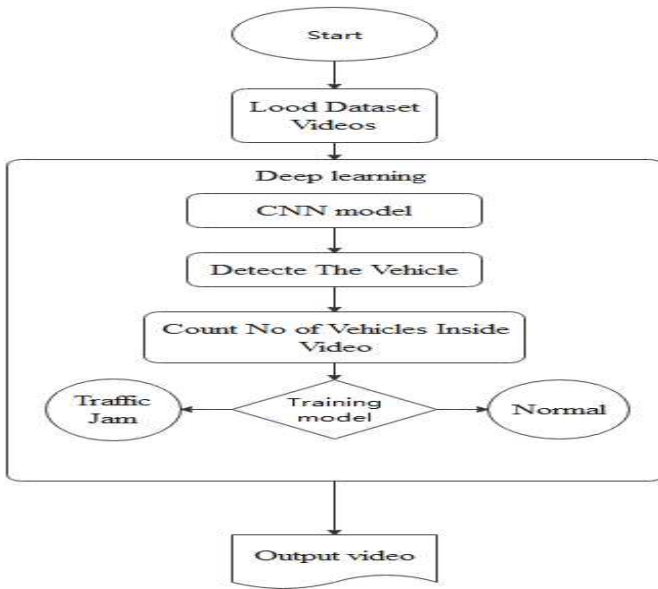


Figure 1. A basic flowchart outlining the primary stages of the method's operation.

A. Deep Learning

Deep learning is a branch of machine learning that draws inspiration from the way information is processed in the human brain. Deep learning algorithms do not require human intervention to design their working mechanisms; instead, they use extensive amounts of data to categorize given inputs. Deep learning is constructed using several layers of algorithms, each of which provides a unique interpretation of the data entered into them.

The sequence of steps involved in machine learning, including pre-processing selection, feature extraction, learning, classification, and feature selection, plays a critical role in determining the performance of the model. Incorrect classification between classes may occur due to the selection of inadequate features. However, in contrast to traditional machine learning methods, deep learning can effectively train on numerous features simultaneously.[6]. Deep learning enables learning and classification in a single snapshot, which has become the performance of deep learning that exceeds human performance in tasks such as image classification.

B. CNN model

CNN was first used and developed in the 1980s. CNN for deep learning is the most used and popular algorithm as compared to its counterpart [7]. When opposed to its equivalent, CNN's primary advantage is the automated feature selection that occurs without human input. Partial interactions, parameter sharing, and equivalent representations are the three main advantages of CNN. In contrast to conventional fully connected networks, CNNs make full use of two-dimensional input data structures, including image signals, by utilizing shared weights and local connections. Due to the reduced number of parameters needed, network performance is accelerated and the training process is made simpler. This is similar to the way individual cells in the visual cortex perceive complete scenes, but just certain portions of them.[8].

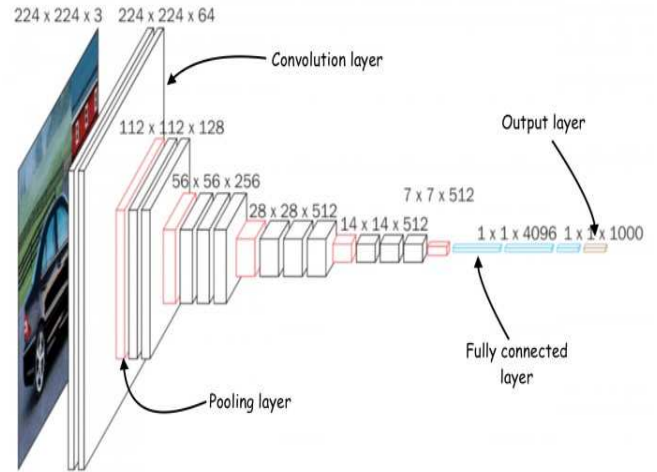


Figure 2. The architecture of CNN.

An CNN This kind of neural network is widely employed in many computer vision applications, including but not limited to image classification, object detection, and segmentation, as shown in Figure 2. CNNs are designed to process inputs that have a grid-like structure, such as images or audio signals.

The main components of a CNN architecture include

- **Convolutional layers:** A convolutional layer performs a convolution operation between its input and a set of learnable filters to produce a set of output feature maps. Every filter is a small matrix of weights that is applied to the input feature map using convolution to generate a single output feature map. The filter slides over the input feature map with a certain stride and performs element-wise multiplication with the input at each location, and then sums the results to produce a single output value[9]. Units like current in amperes and magnetic field in oersteds should not be combined with SI and CGS units. Since equations do not balance dimensionally, this frequently causes confusion. If you must utilize mixed units, make sure you specify the units for each quantity in an equation.

Convolutional layers in Convolutional Neural Networks (CNNs) can be described mathematically using the following equation:

$$y(i,j,k) = f(\text{sum}(w(l,m,n) * x(i+l-1, j+m-1, n) + b(k))) \quad (1)$$

where $y(i,j,k)$ is the activation of the k -th feature map at position (i,j) , $x(i+l-1, j+m-1, n)$ is the input value at position $(i+l-1, j+m-1)$ of the n -th input feature map, $w(l,m,n)$ is the weight of the convolutional filter at position (l,m) for the n -th input feature map and the k -th output feature map, $b(k)$ is the bias term for the k -th output feature map, and $f()$ is the activation function[7].

Convolutional layers have several key advantages over fully connected layers[10]. First, they are able to exploit the spatial correlation between neighboring pixels in an image. Second, they reduce the number of parameters required by a neural network by sharing weights across the image. Finally, they are able to learn features at different levels of abstraction, from simple edges

and corners to more complex structures like faces and animals.

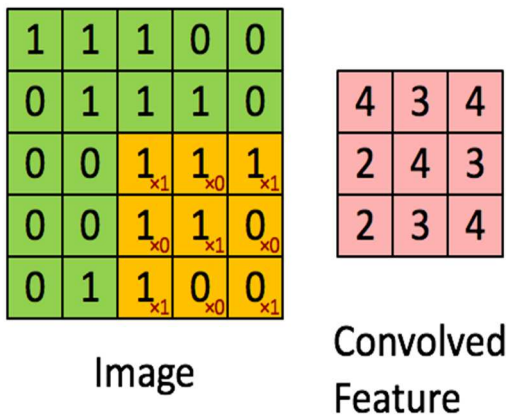


Figure 3 Convolutional layers Architecture.[10]

- **Pooling layers:** Pooling layers play a vital role in Convolutional Neural Networks (CNNs) by decreasing the spatial dimensions of the feature maps created by convolutional layers. These layers are typically added following one or more convolutional layers within a CNN architecture.

The main purpose of pooling is to reduce the size of the feature maps by down-sampling them. This helps to reduce the number of parameters in the network, which can help to prevent over fitting and improve the efficiency of the network. Pooling can also help to make the network more robust to variations in the input, such as changes in the scale, orientation, or position of an object in an image. Max pooling is the most popular kind of pooling; it takes the maximum value from each non-overlapping rectangle or square region in the input feature map and outputs it. Another form of pooling is called average pooling, which outputs the average value for each zone.[11].

During training, the size of the pooling regions and the stride (the distance between adjacent pooling regions) are hyperparameters that can be adjusted to optimize network performance. Typically, smaller pooling regions and larger strides result in more aggressive down-sampling, increasing computational efficiency but potentially resulting in information loss. Conversely, larger pooling regions and smaller strides help preserve more information but can lead to higher computational costs[12].

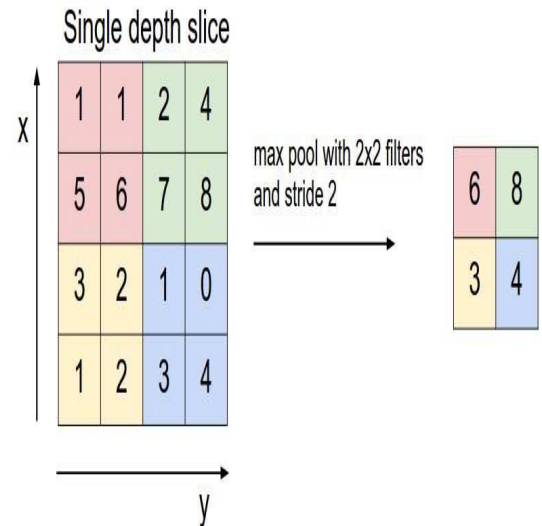


Figure 3. An example that demonstrates the limitations of both max pooling.[12]

Overall, pooling layers are an important tool for reducing the size of feature maps in CNNs and improving the efficiency and robustness of the network.

- **Activation functions:** a critical role in Convolutional Neural Networks (CNNs) by introducing nonlinearity into the network. Nonlinear activation functions enable CNNs to learn complex, nonlinear relationships between the input and output, which is essential for many computer vision tasks.

There are several activation functions that can be used in CNNs, including:

- Rectified Linear Unit (ReLU): s a function that is linear in sections and returns the input value if it is positive, and zero otherwise. ReLU is efficient computationally and has proven to be effective in various CNN architectures [13].

$$f(x) = \max(0, x) \quad (2)$$

Here, 'x' refers to the input to the activation function, and 'f(x)' is the corresponding output.

- Sigmoid: The sigmoid function is beneficial for binary classification tasks as it maps any input to a value ranging from 0 to 1. Nevertheless, for larger input values, the gradient of the sigmoid function may decrease substantially, leading to slow training or even vanishing gradients.[5].

$$f(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x)) \quad (3)$$

Here, 'x' represents the input supplied to the activation function, and 'f(x)' denotes the corresponding output

- c. Tanh: The hyperbolic tangent function is comparable to the sigmoid function but maps inputs to a value ranging from -1 to 1. It is advantageous for classification tasks that involve both positive and negative output values.[14].

$$f(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x)) \quad (4)$$

In this context, 'x' refers to the input supplied to the activation function, and 'f(x)' represents the corresponding output.

- d. Softmax: Softmax is a function that normalizes the output of a CNN so that it can be interpreted as a probability distribution over a set of classes. It is commonly used in classification tasks where the output needs to be one of multiple classes[15].

$$f(x_i) = \exp(x_i) / (\sum(\exp(x_j)) \text{ for } j = 1 \text{ to } N) \quad (5)$$

In this context, 'x_i' represents the input supplied to the activation function for class 'i', 'N' denotes the total number of classes, and 'f(x_i)' represents the output probability for class 'i'.

The selection of an activation function depends on the particular problem being addressed and the structure of the CNN. Typically, ReLU is the preferred activation function due to its straightforwardness and effectiveness. Nonetheless, more intricate architectures may demand the use of other activation functions to attain optimal performance.

CNNs possess numerous advantages over conventional neural networks, such as their capability to take advantage of the spatial correlation between neighboring pixels in an image and their ability to develop hierarchical feature representations at different levels of abstraction. Additionally, they can automatically acquire relevant features from raw data, without requiring manual feature engineering.

C. Traffic jam detection

Traffic jam detection is a computer vision application that uses CNN to detection and classifies traffic conditions in real-time. The goal of traffic jam detection is to identify areas of heavy traffic congestion and alert drivers and traffic management systems in advance so that appropriate measures can be taken to alleviate the traffic flow.

For traffic congestion detection, CNN can be trained on a large dataset of vehicle images to detect and count the number of (cars, buses, and truck) per five seconds to determine traffic, where labels indicate traffic flow conditions (e.g. traffic jam, traffic congestion, or non-traffic) according to equation 6. CNN can then be used to classify new traffic videos in real-time based on their traffic flow conditions.

$$P = \sum_i p / \sum_i PJ + \sum_i pc + \sum_i pn \quad (6)$$

P is the number of more predictions through the system that might be made (Traffic jam, traffic congestion, and normal). J is if the situation is a traffic jam, C is if the situation is traffic congestion, N it is if the situation is normal.

IV. TRAINING AND EVALUATION

Upon training and implementation, a CNN can be integrated into a traffic monitoring system to autonomously detect and classify traffic jam in real-time. The process includes detecting vehicles and counting the number of vehicles within each frame at regular intervals of 'S' seconds (determined by the administrator, for example, S=10 seconds).

If the number of vehicles detected within a given time frame exceeds a predetermined threshold value (which is set by the administrator and may vary based on street size and the amount of vehicle movement during different times of the day, such as rush hour), the system can alert both drivers and traffic management systems to areas of high traffic congestion. This enables appropriate measures to be taken to alleviate congestion and improve the flow of traffic.

We used CNN in Python with the Tensorflow and Civitab libraries to run our system with a video dataset (YouTube-8M). Where fifteen different videos were used, between traffic and normal, the duration of each video is approximately 60 seconds, and you were examined for 10 seconds, and a final result was given for each video, and the condition was determined (traffic jam or normal).

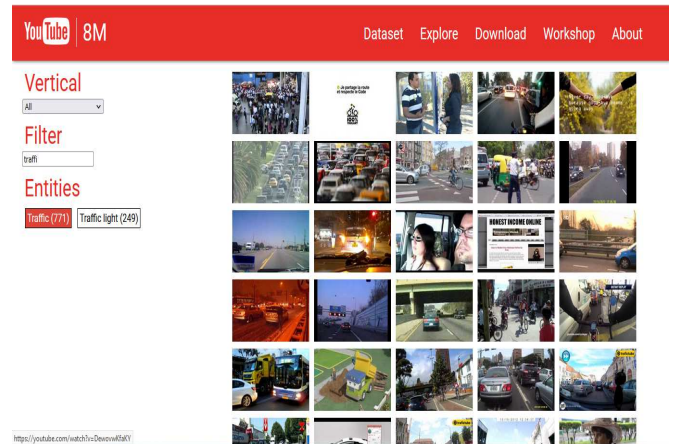


Figure 4.Dataset of youtube8M

To gauge the system's efficiency, the confusion matrix was analyzed. The number of samples that were correctly identified divided by the total number of samples examined was used to calculate the system's precision. The F-score was employed to evaluate the test results since it considers both the system's sensitivity and specificity, which aids in addressing false positives and false negatives. Furthermore, it's crucial to scrutinize the false positive rate (FPR) since it encompasses data that was wrongly categorized as non-traffic congestion.

V. EXPERIMENTAL RESULTS

Different videos depicting various scenarios such as traffic jams, traffic congestion, and normal traffic were used to collect samples. Tests were conducted on each segment of the video according to the specified time frame, to determine whether it belonged to the category of traffic jam, traffic congestion, or normal. Initially, the number of vehicles in each video was determined and a threshold value was set. For example, a threshold value of 15 indicated a traffic jam, while a value of 10 indicated traffic congestion.

While conducting the experiment, the quantity of vehicles appearing in each second of the video was measured and noted in Table 1. If the number of vehicles in a particular frame was equal to or more than the predefined limit, it was classified as either traffic jam or traffic congestion. If the number of cars was below the limit, it was classified as regular traffic. To determine the ultimate status of the video, the outcomes from each second were integrated using equation (6). The final categorization of the video was either traffic jam, traffic congestion, or regular traffic.

The initial evaluation involves the calculation of the number of cars per second from each video. The test obtained for each one second is then presented in Table (1).

Table(1) .The test in one-second intervals

Videos	The status was determined for each second						predicted		Final Status
	NO of car	No of bus	No of truck	Traffic jam	Traffic congestion	Non-Traffic	Traffic%	Non-Traffic%	
TC1	364	2	10	12	7	11	63.3%	36.6%	Traffic Jam
TC2	380	85	6	7	27	1	94.5%	5.5%	Traffic Congestion
NOT3	185	7	13	0	0	29	0.0%	100.0%	Non-traffic
TJ4	416	0	7	14	14	2	93.3%	6.7%	Traffic jam
TJ5	466	0	37	23	0	2	93.4%	6.6%	Traffic jam
NOT6	273	2	11	0	7	23	23.3%	76.7%	Non-traffic
NOT7	135	0	9	0	1	29	3.4%	96.6%	Non-traffic
NOT8	271	1	3	2	16	5	78.2%	21.8%	Traffic congestion
TC9	309	0	26	3	16	11	63.3%	36.7%	Traffic congestion
NOT10	252	3	9	0	6	24	20.0%	80.0%	Non-traffic
NOT11	311	0	16	0	17	14	54.8%	45.2%	Traffic congestion
NOT12	194	5	22	0	2	29	6.4%	93.6%	Non-traffic
TC13	397	2	8	21	5	0	100.0%	0.0%	Traffic jam
TC14	161	11	22	1	11		70.5%	29.5%	Traffic congestion
NOT15	62	0	6	0	0	17	0.0	100.0%	Non-traffic

Here, TC refers to that is traffic congestion video, TJ stands for that is traffic jam video, NOT represents the non-traffic video.

In the second test, the number of vehicles was calculated for every five-second segment in each video, and the results for each segment were recorded as shown in Table (2).

Table(1) .The test in five-second intervals

Videos	The status was determined for each 5 second						predicted		Final Status
	NO of car	No of bus	No of truck	Traffic jam	Traffic congestion	Non-Traffic	Traffic%	Non-Traffic%	
TC1	133	1	2	3	4	4	63.7%	36.3%	Traffic Jam
TC2	145	26	2	3	10	0	100.0%	0.0%	Traffic Congestion
NOT3	75	4	3	0	0	11	0.0%	100.0%	Non-traffic
TJ4	154	0	4	6	4	1	90.8%	9.2%	Traffic jam
TJ5	181	0	10	8	2	1	90.9%	9.1%	Traffic jam
NOT6	103	1	4	0	2	9	18.2%	81.1%	Non-traffic
NOT7	47	0	3	0	0	11	0.0%	100.0%	Non-traffic
NOT8	96	0	1	0	6	2	75.0%	25.0%	Traffic congestion
TC9	111	0	9	0	7	4	63.6%	36.4%	Traffic congestion
NOT10	99	1	5	1	1	9	18.2%	81.8%	Non-traffic
NOT11	110	2	7	1	6	4	63.6%	36.4%	Traffic congestion
NOT12	70	2	6	0	1	10	9.1%	90.9%	Non-traffic
TC13	157	0	4	8	2	0	100.0%	0.0%	Traffic jam
TC14	65	5	6	1	4	1	83.3	16.7%	Traffic congestion
NOT15	23	0	4	0	0	6	0.0	100.0%	Non-traffic

Here, TC refers to that is traffic congestion video, TJ stands for that is traffic jam video, NOT represents the non-traffic video.

The purpose of this test is to assess the workbook's precision when employed with the chosen videos. The performance indicators are obtained by utilizing the confusion matrix. According to the outcomes displayed in Table (3), the mean accuracy is 89.05%.

Table (3). Experimental results

Threshold	Acc	Spe	Sen	F	FPR	FNR	error
One	89.21	79.89	100.0	89.52	0.20	0.0	0.10
Five	88.79	79.38	100.0	89.07	0.20	0.0	0.11
Avg.	89.05	79.63	100.0	89.61	0.2	0.0	0.10

Here, Acc refers to accuracy, Spe represents specificity, Sen sensitivity, FPR the false positive rate, F the F-score, FNR the false negative rate and Avg represents the average.

ACKNOWLEDGMENT

Deep learning-based vehicle detection using CNNs has the potential to significantly contribute to traffic management and reduce congestion on highways. The proposed solution aims to take advantage of CNNs to detect vehicles, calculate their number and classify them (cars, buses, trucks), analyze real-time traffic, and determine whether the traffic condition is

(traffic jam ,traffic congestion or non-traffic). The system was tested on 15 videos ranging from 30 seconds to 60 seconds, where the video status is determined the first time every second and the second time every five seconds, as the average accuracy is 89.5 and the average error rate is 0.10, so that proactive measures are taken to reduce traffic congestion. Through successful implementation, the proposed solution can improve traffic flow, reduce congestion, improve highway transportation efficiency, and provide remarkable service to the public transportation system.

REFERENCES

- [1] F. Knorr, D. Baselt, M. Schreckenberg, and M. Mauve, "Reducing traffic jams via VANETs," *IEEE Transactions on Vehicular Technology*, vol. 61, no. 8, pp. 3490-3498, 2012.
- [2] S. Roy and M. S. Rahman, "Emergency vehicle detection on heavy traffic road from cctv footage using deep convolutional neural network," in *2019 international conference on electrical, computer and communication engineering (ECCE)*, 2019: IEEE, pp. 1-6.
- [3] F. Yang, W. Choi, and Y. Lin, "Exploit all the layers: Fast and accurate cnn object detector with scale dependent pooling and cascaded rejection classifiers," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2129-2137.
- [4] Y. Yang *et al.*, "A fast and effective video vehicle detection method leveraging feature fusion and proposal temporal link," *Journal of Real-Time Image Processing*, vol. 18, no. 4, pp. 1261-1274, 2021.
- [5] M. Roodschild, J. Gotay Sardiñas, and A. Will, "A new approach for the vanishing gradient problem on sigmoid activation," *Progress in Artificial Intelligence*, vol. 9, no. 4, pp. 351-360, 2020.
- [6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436-444, 2015.
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, 2017.
- [8] L. Alzubaidi *et al.*, "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *Journal of big Data*, vol. 8, pp. 1-74, 2021.
- [9] S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a convolutional neural network," in *2017 international conference on engineering and technology (ICET)*, 2017: Ieee, pp. 1-6.
- [10] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015.
- [11] D. Yu, H. Wang, P. Chen, and Z. Wei, "Mixed pooling for convolutional neural networks," in *Rough Sets and Knowledge Technology: 9th International Conference, RSKT 2014, Shanghai, China, October 24-26, 2014, Proceedings 9, 2014: Springer*, pp. 364-375.
- [12] C. Bailer, T. Habtegebrial, and D. Stricker, "Fast feature extraction with CNNs with pooling layers," *arXiv preprint arXiv:1805.03096*, 2018.
- [13] A. F. Agarap, "Deep learning using rectified linear units (relu)," *arXiv preprint arXiv:1803.08375*, 2018.
- [14] B. Karlik and A. V. Olgac, "Performance analysis of various activation functions in generalized MLP architectures of neural networks," *International Journal of Artificial Intelligence and Expert Systems*, vol. 1, no. 4, pp. 111-122, 2011.
- [15] S. Sharma, S. Sharma, and A. Athaiya, "Activation functions in neural networks," *Towards Data Sci*, vol. 6, no. 12, pp. 310-316, 2017.